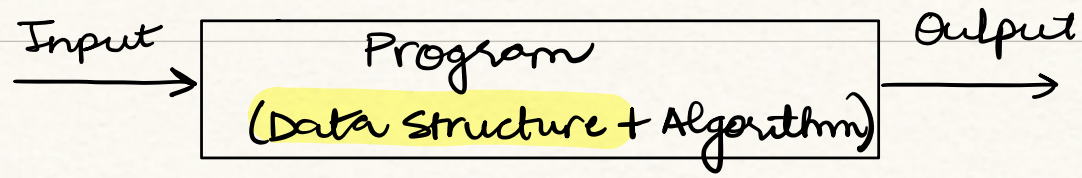
COL 1101

LECTURE 15

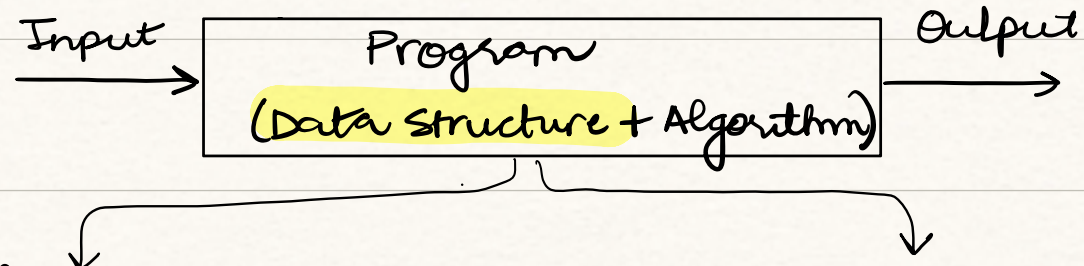
AUGUST 25, 2026

CONCURRENCY

STORY SO FAR



STORY SO FAR



Imperative

#1: Object-oriented programming

Mutable Data + Operations

Encapsulated into objects

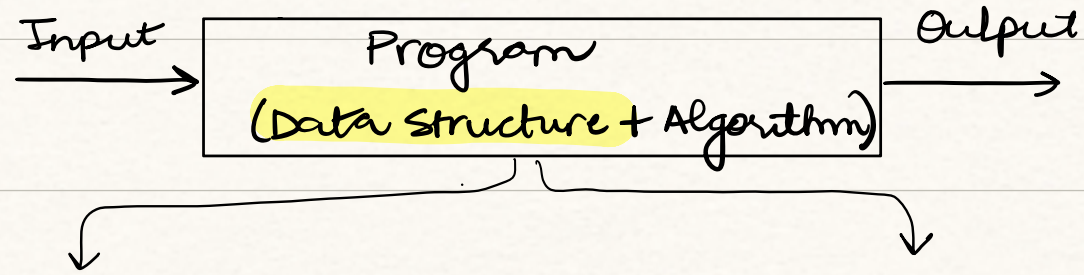
#2: Functional programming

Immutable data + pure functions

$f_n(\dots f_2(f_1(f_0(data))))$

f_i doesn't modify data

STORY SO FAR



#1: Object-oriented programming

Mutable Data + Operations

Encapsulated into objects

#2: Functional programming

Immutable data + pure functions

$f_n(\ldots f_2(f_1(f_0(data))))$

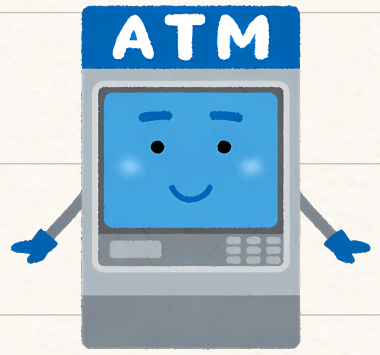
f_i doesn't modify data

Example data structures: Sequence-based

(arrays, linked list, stacks, queues)

ONE ASSUMPTION

A program executes sequentially - one line at a time and in order.




```
balance += 100

balance += 10

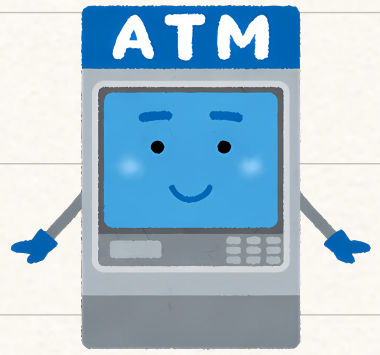
balance -= 50

balance -= 50
```

?

ONE ASSUMPTION

A program executes sequentially - one line at a time and in order.



Run it a million times!

Same output

balance += 100

balance += 10

balance -= 50

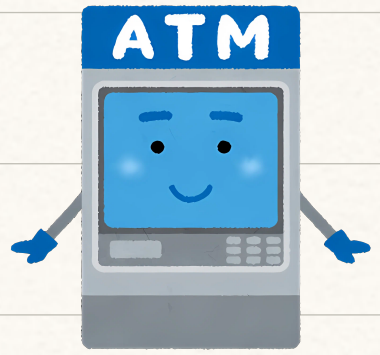
balance -= 50

Single reader-writer

Result is deterministic!

ONE ASSUMPTION

A program executes sequentially - one line at a time and in order.



Run it a million times!

Same output -

balance += 100

balance += 10

balance -= 50

balance -= 50

Single reader-writer

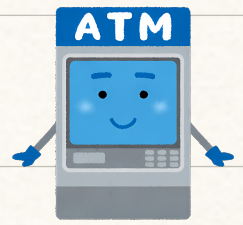
Result is deterministic!

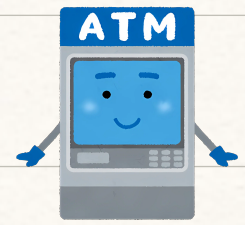
TODAY'S LECTURE

What if two things touch the data at the same time?

TWO CASH MACHINES

same account, same instant!





balance -= 30

balance -= 30

Multiple reader-writers!

CLAIM: This is not always deterministic

Concurrency: when a program has multiple tasks that may overlap in time

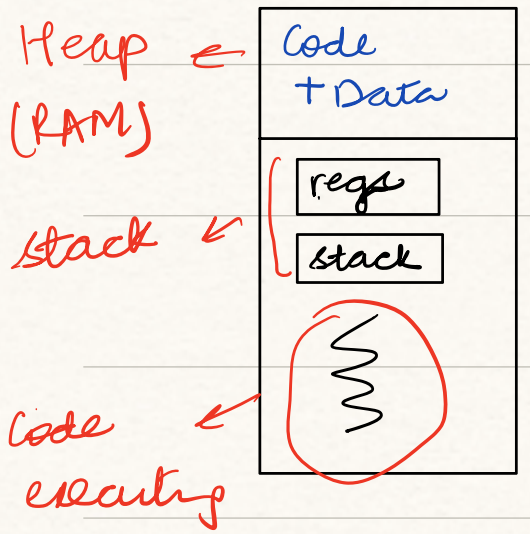
WHY CONCURRENCY ?

Essential in modern programming

- Web sites must handle multiple simultaneous users
- Mobile apps doing processing, GUI, n/w transmission simultaneously
- Background garbage collection in programming languages

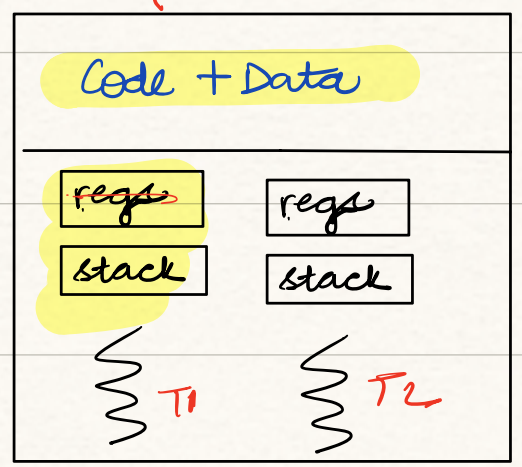
UNIT OF RUNNING AT ONCE : THREADS VS PROGRAMS

Thread



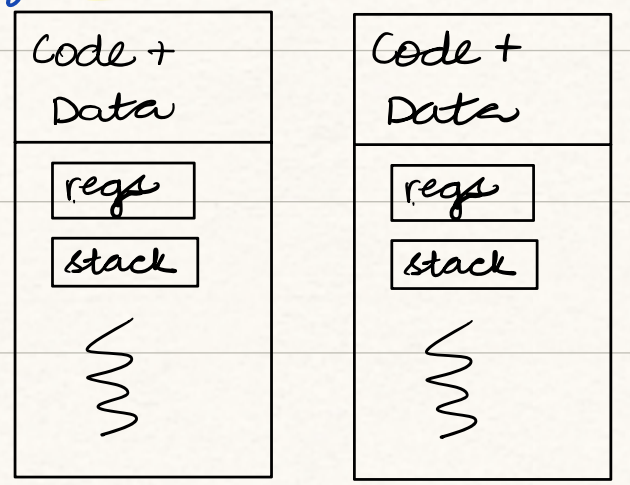
Single-threaded program

heap program



Multi threaded program

share information through files and other mechanisms



ls | grep "collio1"

WHY IT MATTERS -

Thread A

line : balance += amt

(100)

Thread B

balance += amt

Step/time	Thread A	Thread B	Balance
1.	read balance		100
2.	add amt (register)		
3	write balance		200
4		read balance	200
5		add amt	
6		write balance	300

WHY IT MATTERS -

Thread A

line : balance += amt (100)

Thread B

balance += amt

Step	Thread A	Thread B	Balance
1.	read balance		100
2.		read balance	100
3	add amt		100
4		add amt	100
5	write balance		200
6		write balance	2

WHY IT MATTERS

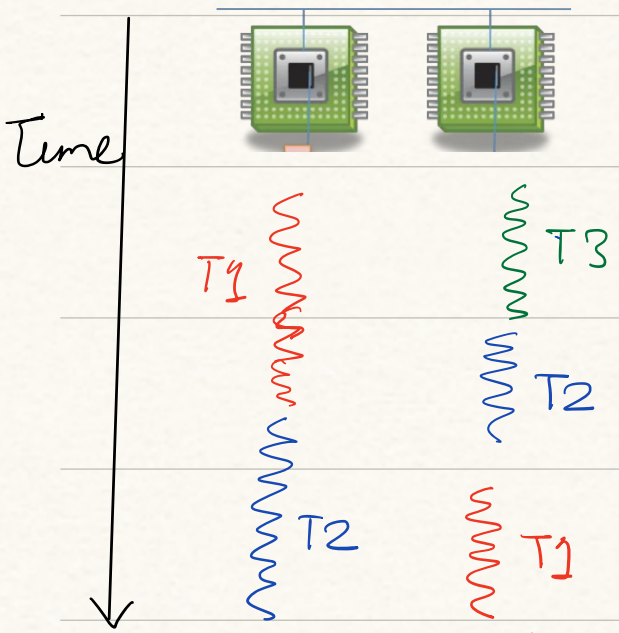
Race condition : correctness depends on relative timing of operations performed by multiple threads

3 essential conditions

1. Shared data / memory
2. Mutable
3. Concurrent access (at least one write)

IS IT ONLY A PROBLEM WITH MULTIPLE PROCESSORS?

Simultaneous execution only in multi-core machines?

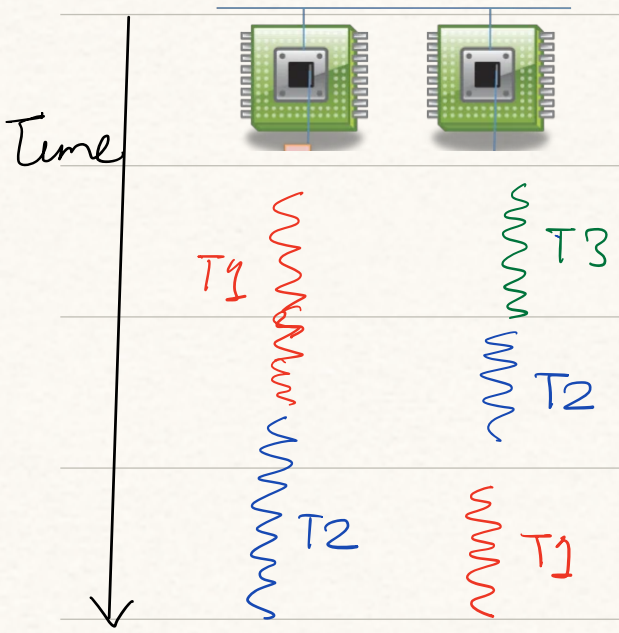


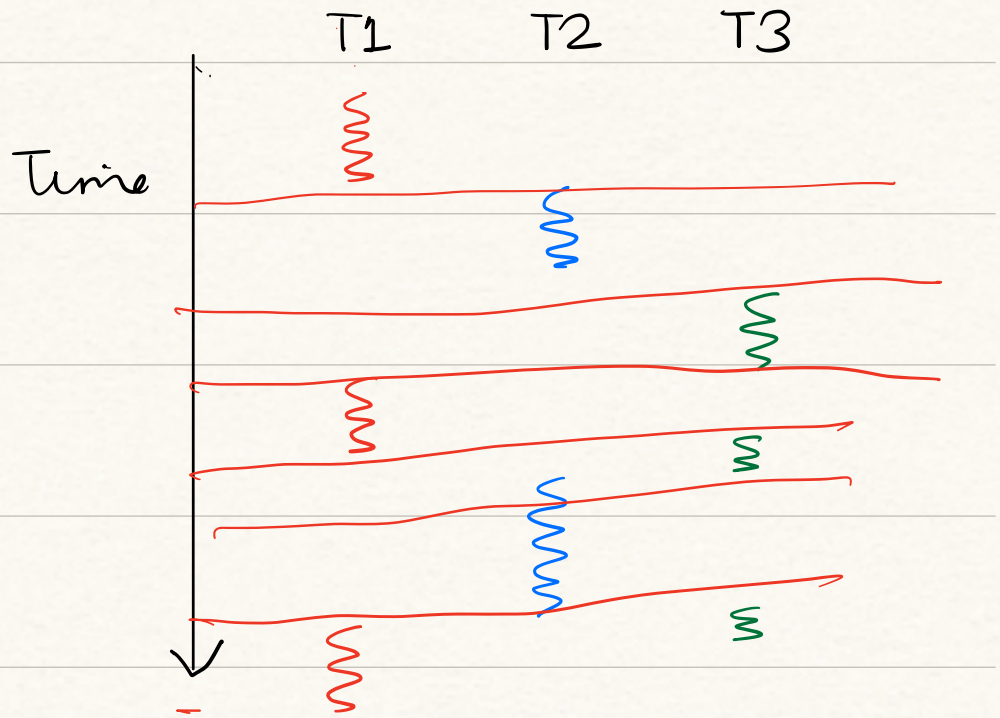
Th A	The B
t=1 read bal	read bal
add	add
write	write

IS IT ONLY A PROBLEM WITH MULTIPLE PROCESSORS?

Simultaneous execution only in multi-core machines

Parallelism





Both are concurrent, only 1st is parallel

HOW TO HANDLE RACE CONDITIONS?

The answer differs for the two programming paradigms.

Case 1: Imperative / Object-oriented programming

HOW TO HANDLE RACE CONDITIONS?

The answer differs for the two programming paradigms.

Case 1: Imperative / Object-oriented programming

↳ can race condition happen here?

Recall the 3 conditions

1. Shared data / memory
2. Mutable
3. Concurrent access. (✗

How do you handle a race condition?

HOW TO HANDLE RACE CONDITIONS ?

The answer differs for the two programming paradigms.

Case 1 : Imperative / Object-oriented programming

Recall the 3 conditions

1. Shared data / memory
2. Mutable
3. Concurrent access.

Add order among the threads through synchronization primitives

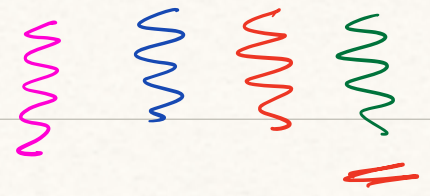
SYNCHRONIZATION PRIMITIVE EXAMPLE : LOCK

```
counter = 0
lock = threading.Lock()
with lock:
    counter += 1
```

lock.acquire()

lock.release()

threads wait for their turn





CRITICAL SECTION

counter += 1

lock

read counter

add 1 counter

write counter

Synchronization Primitive Example: Lock

```
lock = threading.Lock()

class Account:
    def __init__(self, balance):
        self.balance = balance
    def deposit (self, amt) :
        with lock:
            t = self.balance
            time.sleep(0)
            self.balance = t + amt
```

```
def work():
    for _ in range(10000):
        acc.deposit(10)
```

LOCK : TWO IMPORTANT ASPECTS

```
lock = threading.Lock()

class Account:
    def __init__(self, balance):
        self.balance = balance
    def deposit (self, amt) :
        with lock:
            t = self.balance
            time.sleep(0)
            self.balance = t + amt
```

```
def work():
    for _ in range(10000):
        acc.deposit(10)
```

① . What to lock ?

↳ is an important decision

def work
{ work lock:
  for . . .
    acc. deposit (10)

LOCK : TWO IMPORTANT ASPECTS

```
lock = threading.Lock()

class Account:
    def __init__(self, balance):
        self.balance = balance
    def deposit (self, amt) :
        with lock:
            t = self.balance
            time.sleep(0)
            self.balance = t + amt
```

```
def work():
    for _ in range(10000):
        acc.deposit(10)
```

① . What to lock ?

② . Locking creates contention

critical section

